

Engineer Agent Prototype Architecture and Workflow

Full implementation of the 5 agentic design patterns (with real run trajectories)

Companion project: `engineer_agent_prototype v0.1.0`

Patterns covered: Pattern 1-5 (Reflection / Plan and Solve / Tool Use / Multi-Agent / HITL)

July 20, 2026

Contents

- 1 Prototype overview
- 2 Overall architecture
- 3 Main-loop workflow (Pattern 2: Plan and Solve)
- 4 Tool-invocation workflow (Pattern 3: Tool Use)
- 5 Reflection, critique and consultation (Pattern 1 / Pattern 4)
- 6 End-to-end execution sequence (real run trajectory)

1 Prototype overview

This prototype (`engineer_agent_prototype`) implements a runnable engineering-domain multi-agent system per the "Engineer Large Model (Open Source)" architecture diagram, **landing all five design patterns**: P1 Reflection (generator-critic self-critique loop + lesson persistence), P2 Plan and Solve (LangGraph plan-execute-replan main loop), P3 Tool Use (MCP-style controlled tool layer), P4 Multi-Agent Collaboration (declarative specialist agents + `team.ask` inter-agent consultation), P5 Human-in-the-Loop (permission engine + high-risk blocking + acceptance gate). With the "outdoor energy-storage power supply design" example task, the system runs the full flow — task decomposition, specialist execution, check failure, reflection, consultation, parameter tuning, re-check, critique-and-revise, report generation, high-risk blocking, acceptance — reproducible offline with no API Key, and switchable to a real LLM in one flag. The five-pattern framework is decoupled from any specific task: with zero code changes, a new "smart meter" task was verified end-to-end (the real model autonomously planned 5 steps, all passed); task dependence exists only in the declarative agent definitions and the tools' simulated data, both replaceable on demand.

Table 1 Technology stack and reference projects

Layer	Selection	Reference project	Pattern
Plan-execute main loop	LangGraph Plan-and-Execute	langchain-ai/langgraph	P2
Plan-optimization ideas	DAG parallelism / replanning (architectural borrowing)	SqueezeAILab/LLMCompiler	P2
Tool access layer	MCP-style controlled invocation layer (minimal implementation)	modelcontextprotocol/servers	P3
Self-critique loop	generator-critic two nodes + lesson persistence	LangGraph Reflection tutorial / Reflexion / Self-Refine / CRITIC	P1
Inter-agent consultation	<code>team.ask</code> meta-tool + declarative specialist agents	crewAI / AutoGen / sst/opencode	P4
Permission model	allow / ask / deny + wildcards + acceptance gate	sst/opencode / LangGraph interrupt / AG-UI	P5
LLM abstraction	MockLLM (offline) / OpenAI-compatible API	— (vLLM/Ollama open-source bases supported)	—

2 Overall architecture

The prototype has five layers: access layer (CLI) → main-loop layer (LangGraph state graph) → agent layer (declarative specialist Engineers) → controlled tool layer (MCP style) → engineering environment layer. Every tool call must pass through the controlled tool layer; the reflection module spans both execution and replanning, as shown in [Figure 1](#).

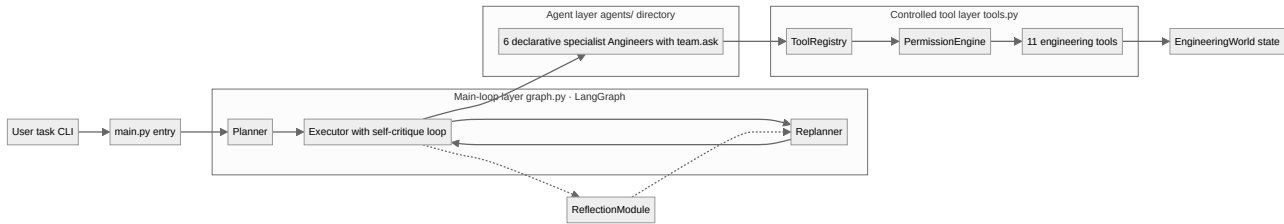


Figure 1 Overall architecture (five layers; dashed lines are reflection data flows)

The correspondence between each layer and the original architecture diagram is shown in [Table 2](#) — every functional block on the diagram has a landing spot in code.

Table 2 Architecture-diagram modules → prototype code mapping

Diagram module	Code location	Implementation
Task Decomposition	<code>graph.py</code> <code>planner_node</code>	LLM decomposes the task into a {agent, instruction} step list
Actions / bottom specialist-Engineer bus	<code>graph.py</code> <code>executor_node</code> + <code>agents/* .md</code>	Each step routes to its specialist agent, driving a ReAct mini-loop
Decisions	replanner conditional edges	Three-way branch: advance / replan / finish
Space Control	<code>MAX_STEP_ITERS=8</code> / <code>MAX_REPLANS=2</code> / <code>MAX_CRITIQUES=2</code>	Per-step circuit breaker + replan cap + critique-round cap, preventing runaway loops
Reflection (self-critique)	<code>graph.py</code> <code>executor critique loop</code> + <code>llm.critique()</code>	final is reviewed by the critic first; revise rewrites with feedback (at most 2 rounds)
Multi-Agent collaboration	<code>graph.py</code> <code>_consult</code> + <code>team.ask</code> meta-tool	Agents consult each other on professional questions; no nesting, 3-step cap, fully traced
HITL acceptance gate	replanner done branch <code>final_acceptance</code>	After all steps complete, manual acceptance is required to end the flow

Engineer Tools	<code>tools.py</code> ToolRegistry	Controlled invocation layer: declarative schema + permission wrapper + logging
Engineer Knowledge Graph	<code>kg_query</code> tool	Engineering knowledge-graph query (simulated, read-only)
Data Preparation and Transform	<code>data_transform</code> tool	Sensor/CAD data cleaning and transformation (simulated)
Debug and Refine	<code>reflection.py</code> + replanner_node	Runtime introspection + plan-level correction
Optimization	<code>optimization.set_param</code> tool	Modifies design variables (ask permission)
Writing Report / Apply Results	<code>report.write</code> / <code>production.apply</code>	Report writes need confirmation; production apply is denied by default
LLM	<code>llm.py</code>	MockLLM (offline rules) and OpenAI-compatible API, interchangeable

Correspondence of the bottom green bus: Structure Design / Chip Design / PCB CAM / Safety / Discharge Monitoring each have their own agent file; the **Inspection Engineer's** duty lands on the `inspection.query` tool (held by the Discharge Monitoring agent); **Industry Engineer** and "Etc..." are not yet implemented — dropping one more md file into `agents/` completes the extension without touching the main loop, exactly the extension path the diagram reserves.

```

engineer_agent_prototype/
├─ main.py                # CLI entry (--task / --auto-approve / --real / --upload)
├─ agents/                # 6 declarative specialist agents (one md file = one
Engineer)
│   ├── structure_design.md  chip_design.md  pcb_cam.md
│   ├── safety.md           discharge_monitoring.md  chief.md
├─ engineer/
│   ├── graph.py            # LangGraph Plan-Execute-Replan main graph (P2 core)
│   └── tools.py            # MCP-style tool registry + controlled invocation layer
(P3 core)
│   ├── permissions.py      # allow/ask/deny permission engine (P5)
│   ├── reflection.py       # periodic self-check + lesson persistence (P1)
│   ├── upload.py           # --upload data intake: design JSON / monitoring CSV
│   ├── agents_loader.py    # declarative agent loader
│   └── llm.py              # LLM abstraction: MockLLM / OpenAILLM
└─ docs/                  # technical selection rationale (Chinese + English) and
architecture PDFs

```

3 Main-loop workflow (Pattern 2: Plan and Solve)

The main loop is a LangGraph-compiled state graph with three nodes and one conditional edge, following the official Plan-and-Execute paradigm: the **Planner** produces an explicit plan (step list), the **Executor** dispatches specialist agents step by step, and the **Replanner** decides "advance / correct / finish" after each step, as shown in [Figure 2](#). The plan is an explicit data structure and failed steps are re-queued — borrowing LLMCompiler's "plans are correctable" idea.

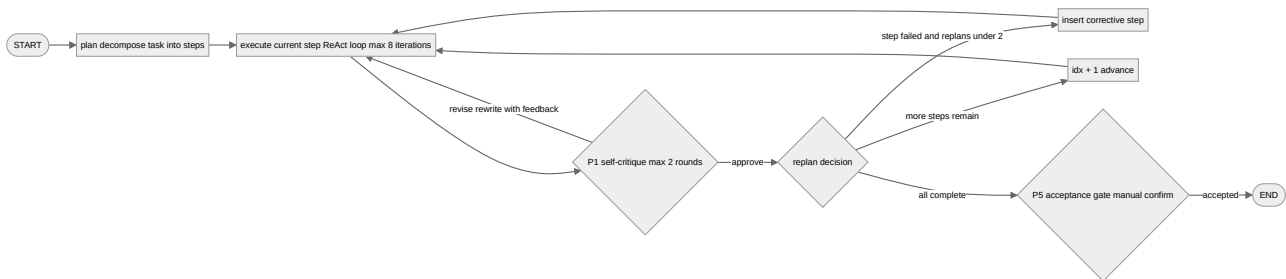


Figure 2 Main-loop state graph (Planner → Executor → Replanner, with P1 critique loop, P5 acceptance gate and Space Control circuit breakers)

Table 3 Main-loop node responsibilities

Node	Input → output	Responsibility
plan	task text → plan step list	Calls <code>llm.decompose()</code> : MockLLM routes by keywords to specialist agents; a real model generates a JSON plan from the agent roster; a chief summary step is appended automatically
execute	current step → execution result	Loads the step's agent (system prompt + authorized tool list + permission overrides) and enters a ReAct mini-loop: <code>llm.act()</code> decision → controlled tool call → result into scratchpad; after the agent gives final, <code>llm.critique()</code> self-critiques it — revise rewrites with feedback (at most 2 rounds), approve finalizes the step; each step's instruction is injected with summaries of the last 4 prior results as context
replan	latest result + lessons → routing decision	If the step failed and the replan cap is not reached, inserts a corrective step carrying the latest lesson back into the queue; otherwise advances; when all steps complete, triggers the P5 acceptance gate (<code>final_acceptance</code> , manual confirmation) and ends

Space Control (runaway prevention). Corresponding to the Space Control block on the diagram, the prototype sets three hard constraints: at most `MAX_STEP_ITERS=8` tool

calls per step (prevents spinning inside one step), at most `MAX_REPLANS=2` replans per run (prevents "fail-retry" infinite loops), and at most `MAX_CRITIQUES=2` self-critique rounds per step (prevents "critique-revise" infinite loops). A unit test injecting a "never converge" fault verifies the system terminates cleanly after 3 executions and 2 replans — no infinite loop.

4 Tool-invocation workflow (Pattern 3: Tool Use)

Agents never touch any external system directly. Every invocation goes through the controlled layer `ToolRegistry.invoke()`, which always executes the five-stage pipeline "schema validation → permission matching → execution → logging → reflection", as shown in [Figure 3](#). This is exactly the article's definition of Tool Use: the model may only work through controlled interfaces that declare purpose, parameters and output format; permissions converge at the tool level rather than being entrusted to the model.

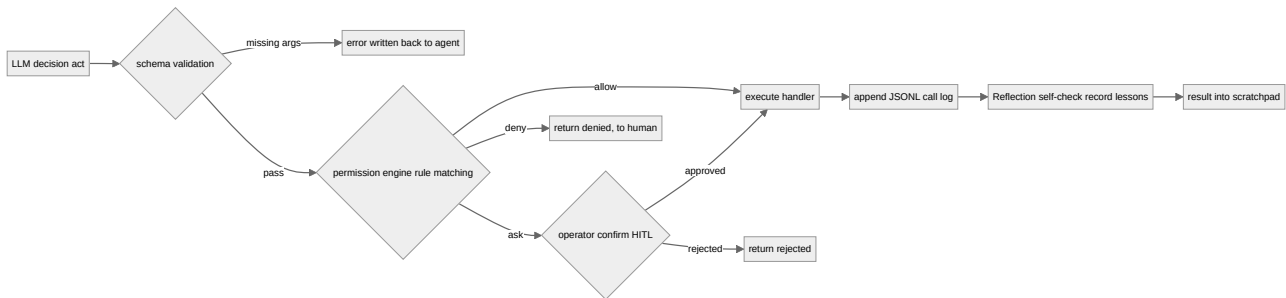


Figure 3 Controlled pipeline of a single tool call (with the three HITL branches)

The permission engine follows `opcode`: rules match by wildcards one by one, **last match wins**, and permission overrides in agent files are appended after the global rules (agent level takes precedence). The global default rules are shown in [Table 4](#).

Table 4 Global permission rules (`permissions.py`)

Tool pattern	Decision	Rationale
<code>kg_query /</code> <code>inspection.query /</code> <code>discharge_sensor.*</code>	allow	Read-only queries: knowledge graph, inspection records, discharge sensors
<code>team.ask</code>	allow	Inter-agent consultation meta-tool: touches no external system, only drives one read-only answer from the consulted agent
<code>data_transform /</code> <code>structure_calc.* /</code> <code>cam_check.* / chip_sim.*</code>	allow	Pure internal computation and simulation, no side effects

<code>optimization.*</code>	ask	Modifies design parameters; engineer confirmation required
<code>safety_audit.*</code> / <code>report.write</code>	ask	Safety-audit conclusions and persisted files need confirmation
<code>production.*</code>	deny	Applying to production is high-risk; denied by default in the prototype and only a human rule change can allow it

Table 5 Tool inventory (11 engineering tools + 1 inter-agent consultation meta-tool, MCP-style schema declarations)

Tool	Purpose	Read-only	Authorized agents
<code>kg_query</code>	Engineering knowledge-graph query (materials/processes/components)	Yes	structure_design, chip_design, pcb_cam
<code>data_transform</code>	Data cleaning and transformation	No	safety
<code>structure_calc.run</code>	Structural strength check	No	structure_design
<code>chip_sim.run</code>	Chip power/timing/DRC simulation	No	chip_design
<code>cam_check.run</code>	PCB current-carrying/trace-width CAM check	No	pcb_cam
<code>optimization.set_param</code>	Optimize design parameters	No	pcb_cam
<code>safety_audit.run</code>	Safety/thermal/protection audit	No	safety
<code>inspection.query</code>	Quality inspection record query	Yes	discharge_monitoring
<code>discharge_sensor.read</code>	Discharge monitoring data read	Yes	discharge_monitoring
<code>report.write</code>	Generate engineering report	No	chief
<code>production.apply</code>	Apply change to production (high-risk)	No	chief (blocked by deny)

<code>team.ask</code> (meta-tool)	Inter-agent consultation: ask another specialist and bring back a structured answer	Yes	All 6 agents (no nesting, 3-step cap)
--------------------------------------	---	-----	---------------------------------------

5 Reflection, critique and consultation (Pattern 1 / Pattern 4)

The Debug and Refine block on the diagram lands in three layers: **result-level self-critique** by the generator-critic loop inside the Executor (structured after the LangGraph Basic Reflection tutorial; ideas from Reflexion / Self-Refine / CRITIC); **runtime introspection** by the `ReflectionModule` (after hermes-agent's kernel self-check loop); and **plan-level correction** by the Replanner node. The three connect through the shared lessons list and scratchpad: critique feedback is written back to drive revision, and lessons precipitated during execution feed the corrective steps of replanning.

5.1 The self-critique loop (P1 core)

After an agent gives its final, it is not accepted directly: `llm.critique()` switches the same LLM into a reviewer role to check the conclusion against the task instruction for completeness and evidence sufficiency, returning `approve` or `revise + feedback`. On revise, the feedback enters the scratchpad and the agent regenerates with it, at most `MAX_CRITIQUES=2` rounds. In k3 real runs this mechanism repeatedly did substantive work: it caught the structure agent's "safety factor 1.37 vs 1.52 contradiction", criticized the Safety agent's conclusion as "insufficient evidence, do not approve for now", and pointed out that the chief's first report draft "lacked a structured summary of specialist results, risk grading and change disposition", producing a revised version.

5.2 Lesson persistence and periodic self-check

The `ReflectionModule` fires by rule after every tool call, solidifying anomalies and experience into structured lessons, reused within the same step and during replanning. The trigger rules are shown in [Table 6](#).

Table 6 ReflectionModule trigger rules (reflection.py)

Trigger	Action	Example (from real runs)
Tool result status = fail	Persist one structured lesson	"tool cam_check.run failed: trace width 0.20mm below required 0.30mm → retry after strategy adjustment"

status = error (schema validation failed)	Record "fix args per tool schema"	"tool structure_calc.run call error: missing args ['model'] → fix args per tool schema"
status = denied / rejected	Record "such operations need manual approval"	"tool production.apply blocked by human/policy: such operations require manual approval"
Every 3 tool calls	Periodic self-check, assess whether the trajectory is healthy	"periodic self-check (3rd tool call): trajectory normal, continue"

5.3 Inter-agent consultation (P4 core)

Multi-agent collaboration goes beyond task dispatch: every agent also carries the `team.ask` meta-tool to consult another agent on cross-disciplinary questions. Consultation is orchestrated by the main graph (the consulted party runs an independent 3-step mini-loop, nested `team.ask` is forbidden, permission checks still apply, and calls are traced into the JSONL log) — avoiding the runaway risk of CrewAI-style free-form dialogue, and closer to AutoGen's controlled message passing and opencode's declarative sub-agent invocation. In a k3 real run the model spontaneously consulted a nonexistent "`certification_engineer`"; the trio of structured error feedback + lesson persistence + replanning caught it cleanly, and the agent then consulted the real Safety agent — a live validation of the consultation mechanism's fault tolerance.

Taking the PCB CAM step of the example task as an example, the full closed loop is: `cam_check.run` (0.20mm) → fail → **reflection records a lesson** → `team.ask` consults the Safety agent (P4 consultation: confirm the fix direction introduces no safety risk) → the agent decides to tune via `optimization.set_param` (0.35mm, ask permission, operator approves) → re-check `cam_check.run` (0.35mm) → pass → **P1 self-critique** review before finalizing. Note the real state dependency between tools: `set_param` modifies the trace width in `EngineeringWorld`, so the re-check result changes accordingly — making the "check-consult-optimize-re-check" loop reproducible real logic rather than a hardcoded script.

6 End-to-end execution sequence (real run trajectory)

[Figure 4](#) shows the sequence of the most critical PCB CAM step in the example task (with P4 consultation, HITL confirmation, P1 critique and the P5 acceptance gate), and [Table 7](#)

lists all 12 tool calls of one complete real run (`python main.py --task "..." --auto-approve`).

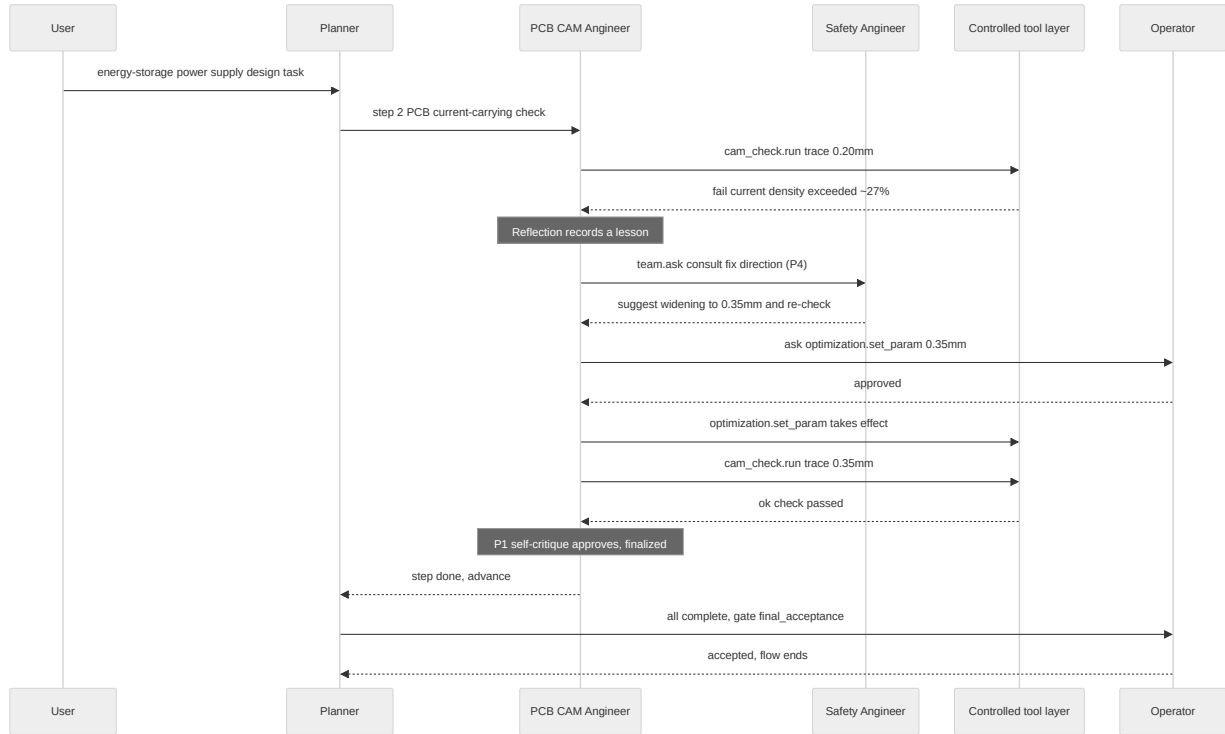


Figure 4 PCB CAM step sequence (check fail → reflection → P4 consultation → HITL tuning → re-check → P1 critique finalize → P5 acceptance)

Table 7 Full execution trajectory of the energy-storage example task (real run, 12 tool calls incl. 1 inter-agent consultation)

#	Step (agent)	Tool call	Permission	Result
1	Structural check (structure_design)	<code>kg_query</code> aluminum 6061	allow	material parameters hit
2		<code>structure_calc.run</code> battery-bay beam	allow	stress 182MPa, safety factor 1.37, pass

6 End-to-end execution sequence (real run trajectory)

3	CAM check (pcb_cam)	cam_check.run 0.20mm	allow	fail current density ~27% over → lesson recorded
4		team.ask consult safety	allow	P4 consultation: widen to 0.35mm and re-check thermals
5		optimization.se t_param 0.35mm	ask→approved	design parameter updated
6		cam_check.run 0.35mm	allow	pass → finalized after P1 critique review
7	Safety audit (safety)	data_transform thermal monitoring data	allow	12,480 rows cleaned
8		safety_audit.ru n whole-unit safety	ask→approved	pass, 1 rectification suggestion
9	Discharge monitoring (discharge_monitoring)	discharge_senso r.read pack-01	allow	SOC 62%, 41.5°C, normal
10		inspection.quer y finished goods	allow	first-pass yield 98.6%
11	Summary (chief)	report.write engineering report	ask→approved	report persisted (first draft revised after P1 critique)
12		production.appl y trace-width change	deny	blocked by permission policy, routed to manual approval

Each run produces two traceable artifacts: `output/engineering_report.md` (automatically aggregated specialist results and lesson records) and `output/tool_calls.jsonl` (12 timestamped call-log entries, including the team.ask consultation trace). The whole trajectory covers all five design patterns: P1 (self-critique loop + lesson persistence), P2 (plan-execute-replan main loop), P3 (controlled tool invocation), P4 (task dispatch + team.ask inter-agent consultation), P5 (ask confirmations, deny blocking and the acceptance gate).